

Investments SSO

Let a Blossom member tap **Investments** and land in the investing app already signed in — no second signup, no second password.

PROTOCOL

OpenID Connect · Authorization
Code + PKCE

OUR ROLE

Relying Party (client)

STACK

Any — Python, Node, Java, .NET, Go,
Ruby

YOUR ROLE

OpenID Provider (identity provider)

YOUR EFFORT

Register one client · add one link

AUDIENCE

Blossom backend + frontend
engineers

Contents

01	What we are building	— and what your members will experience
02	The integration on one page	— three values, one URL, one link
03	What your provider must support	— capabilities and libraries
04	The claims we read	— and the two that must be right
05	Register Surmount as a client	— step by step
06	Add the link	— one anchor tag
07	How the flow works	— so you can debug it
08	Test it before we meet	— a self-service checklist
09	What you do not need to build	
10	What we do on our side	— for your security review
11	Frequently asked questions	
12	Before go-live	— non-technical prerequisites

01 What we are building

THE MEMBER'S EXPERIENCE

- A member is signed in to Blossom and using it normally.
- They tap **Investments** in the sidebar.
- They land in the Blossom-branded investing app, **already signed in**.
- They type nothing. No login screen, no signup form, no password.
- If they have never invested before, an account is created silently from what Blossom already knows.
- If they have been before, it is the same account every time — even if they have since changed their email.

HOW THAT IS POSSIBLE

Instead of asking the member who they are, the investing platform **asks Blossom directly, server to server**, and Blossom vouches for them with a **cryptographically signed statement**.

- ✗ Blossom never hands over a password.
- ✗ Surmount never sees one — only a claim it can verify against your public key.

THE FOUR PARTS

Blossom FE

Your web app. Holds one link and nothing else.
yours

Blossom BE

Your API **and** your OpenID Provider. Owns the login, the password, the identity.
yours – the work described in this guide

Whitelabel

The Blossom-branded investing app. Looks like Blossom, runs on Surmount. No sign-in of its own.
ours

Surmount BE

The brokerage backend. Accounts, KYC, custodians, trades. Your members never see this name.
ours

THE GOOD NEWS

This is all standard OpenID Connect. **Your stack does not matter** — Python, Node, Java, .NET, Go and Ruby all have mature libraries that do the protocol work. If Blossom already runs an identity provider for staff SSO or a mobile app, most of this is already done and the remaining work is registering one more client.

02 The integration on one page

You give us three values. We give you one URL. You add a link.

YOU SEND US

VALUE	EXAMPLE	HOW TO SEND IT
issuer	https://id.blossom.com	Email is fine
client_id	surmount-investing	Email is fine
client_secret	(32+ random bytes)	Secrets manager or 1Password. Never email or Slack.

WE GIVE YOU

VALUE	WHAT IT IS
https://api.surmount.ai/api/sso/blossom/callback/	Our redirect URI — allow-list it exactly
https://api.surmount.ai/api/sso/blossom/start/	The link for your sidebar

YOU CONFIGURE, ON YOUR SIDE

- Allow-list our redirect URI — **exact string match, no wildcards**.
- Mark the client **trusted** so it skips the consent screen.
- Optionally register a post-logout redirect: `https://app.blossom.com/dashboard`.

THERE IS NO SDK

Nothing to install. No API for your frontend to call. No webhook to receive. Everything happens over standard OIDC endpoints your library already provides.

03 What your provider must support

We read all of this from your discovery document at `{issuer}/.well-known/openid-configuration`, so your endpoint URLs stay yours to change without breaking us.

CAPABILITY	REQUIRED VALUE	WHY
<code>response_types_supported</code>	<code>code</code>	Authorization Code flow. Tokens never touch the browser
<code>grant_types_supported</code>	<code>authorization_code</code>	We do not use refresh tokens against your provider
<code>code_challenge_methods_supported</code>	<code>S256</code>	PKCE. We always send it — please require it, and do not accept <code>plain</code>
<code>id_token_signing_alg_values_supported</code>	<code>RS256</code> or better	Asymmetric. You keep the private key; we only ever hold the public one
<code>jwks_uri</code>	<code>public</code> , cacheable	Where we fetch your public key to verify signatures
<code>scopes_supported</code>	<code>openid</code> <code>profile</code> <code>email</code>	What we ask for
<code>token_endpoint_auth_methods</code>	<code>client_secret_basic</code>	We authenticate with HTTP Basic at your token endpoint

YOU DO NOT NEED

- ✗ Implicit or hybrid flow
- ✗ Refresh tokens
- ✗ Dynamic client registration
- ✗ Token introspection or revocation endpoints
- ✗ Client-initiated backchannel authentication (CIBA)

LIBRARIES, BY STACK

You should write almost no security code. Configure a library instead:

STACK	LIBRARY
Python	authlib
Python / Django	django-oauth-toolkit
Node	node-oidc-provider
Java / Spring	Spring Authorization Server
.NET	Duende IdentityServer, OpenIddict
Go	fosite
Ruby	doorkeeper + OIDC extension

A WORKING REFERENCE IS AVAILABLE

We built a complete OpenID Provider in Python (authlib) with 33 tests while developing this integration. It is a **conformance target to develop against** — run it locally, point a test client at it, compare behaviour. For production, use the library for your own stack directly rather than porting our code.

04 The claims we read

From the ID token, or from `/userinfo` if you keep the ID token minimal.

CLAIM		WHAT WE DO WITH IT
sub	required	The permanent link between your member and their Surmount account
email	required	Creating the account, and matching an existing one
email_verified	important	If <code>true</code> , we skip our own verification email
given_name	preferred	Their name in the app, and a KYC prefill
family_name	preferred	As above
picture	optional	Their avatar. Must be an <code>https</code> URL

Two claims that must be right

1 • SUB MUST BE IMMUTABLE AND NEVER REUSED

- **It must never change** for a given human — not on email change, not on plan change, not on a profile edit.
- **It must never be reused** for a different human — including after a user is deleted.

We store `sub` and match on it forever. We deliberately do **not** match on email, because emails get changed and recycled — and matching a recycled email would hand one person another person's brokerage account.

A UUID primary key gives you both properties for free. A username, an email address, or a sequential ID that gets reissued after deletion gives you neither.

2 • EMAIL_VERIFIED MUST MEAN WHAT IT SAYS

If you send `true`, we take your word for it and skip our own verification email. If Blossom lets someone set an email address without proving they own it — for example a profile page that does not re-verify on change — that claim becomes a route to account takeover on our side.

If in doubt, send `false`. We will verify it ourselves and nothing breaks.

EXAMPLE ID TOKEN PAYLOAD

WHAT WE EXPECT TO RECEIVE

```
{
  "iss": "https://id.blossom.com",
  "sub": "8c14b7e2-0000-4000-8000-9d3ef1a27c04",
  "aud": ["surmount-investing"],
  "exp": 1786029000,
  "iat": 1786025400,
  "auth_time": 1786025380,
  "nonce": "<the value we sent on the authorize request>",
  "name": "Nadia Halim",
  "given_name": "Nadia",
  "family_name": "Halim",
  "email": "nadia@example.com",
  "email_verified": true,
  "picture": "https://id.blossom.com/avatar/8c14b7e2.svg"
}
```


05 Register Surmount as a client

1 Create a confidential client in your provider

Via your admin UI, a management command, or a migration — whatever your library provides.

SETTING	VALUE
Client type	Confidential (has a secret)
Client ID	surmount-investing
Grant types	authorization_code
Response types	code
Scopes	openid profile email
Token endpoint auth	client_secret_basic
ID token alg	RS256
PKCE	Required, S256 only

2 Allow-list our redirect URI — exactly

EXACT STRING MATCH · NO WILDCARDS · NO PREFIX MATCHING

`https://api.surmount.ai/api/sso/blossom/callback/`

Loose redirect-URI matching is the most reliably exploited bug in OAuth deployments: anything looser lets an attacker append their own host and receive the authorization code. Note the trailing slash — it is part of the string.

Optionally, a post-logout redirect: `https://app.blossom.com/dashboard`

3 Mark the client trusted, so it skips the consent screen

Different libraries name this differently — `skip_authorization`, `is_trusted`, "first-party", "pre-authorized". Find yours and enable it.

Surmount is first-party from the member's point of view: Blossom handing a Blossom member to Blossom's own investing product. A consent prompt asking whether Blossom may share their name with Blossom is noise, and it teaches members to click through consent screens.

4 Send us the three values

- `issuer` and `client_id` — email is fine, they are not secret.
- `client_secret` — **via your secrets manager or 1Password.**

Most providers show the secret **once** and store only a hash, so it cannot be looked up later — only replaced. Capture it carefully.

06 Add the link

BLOSSOM FE · YOUR SIDEBAR

```
<a href="https://api.surmount.ai/api/sso/blossom/start/">Investments</a>
```

THIS MUST BE A PLAIN ANCHOR

- ✗ Not `fetch()` or `XMLHttpRequest`
- ✗ Not a popup or a new window
- ✗ Not an `iframe`

The hand-off is a chain of redirects that must carry cookies across origins — your session cookie on the way to `/authorize`, ours on the way back. Only a **top-level navigation** does that. An XHR would be blocked cross-origin, could not follow the chain, and could not hold the cookies.

WHY THE LINK POINTS AT SURMOUNT, NOT AT BLOSSOM

- The flow must begin wherever the security values (`state` , `nonce` , PKCE verifier) are generated and stored — and that is the relying party, i.e. us.
- If Blossom started it, you would have to generate *our* state and know *our* client configuration. That is backwards.
- This is the standard **RP-initiated** pattern, the same shape used by Spring Security, NextAuth and django-allauth.

NOTHING HAPPENS AT CLICK TIME

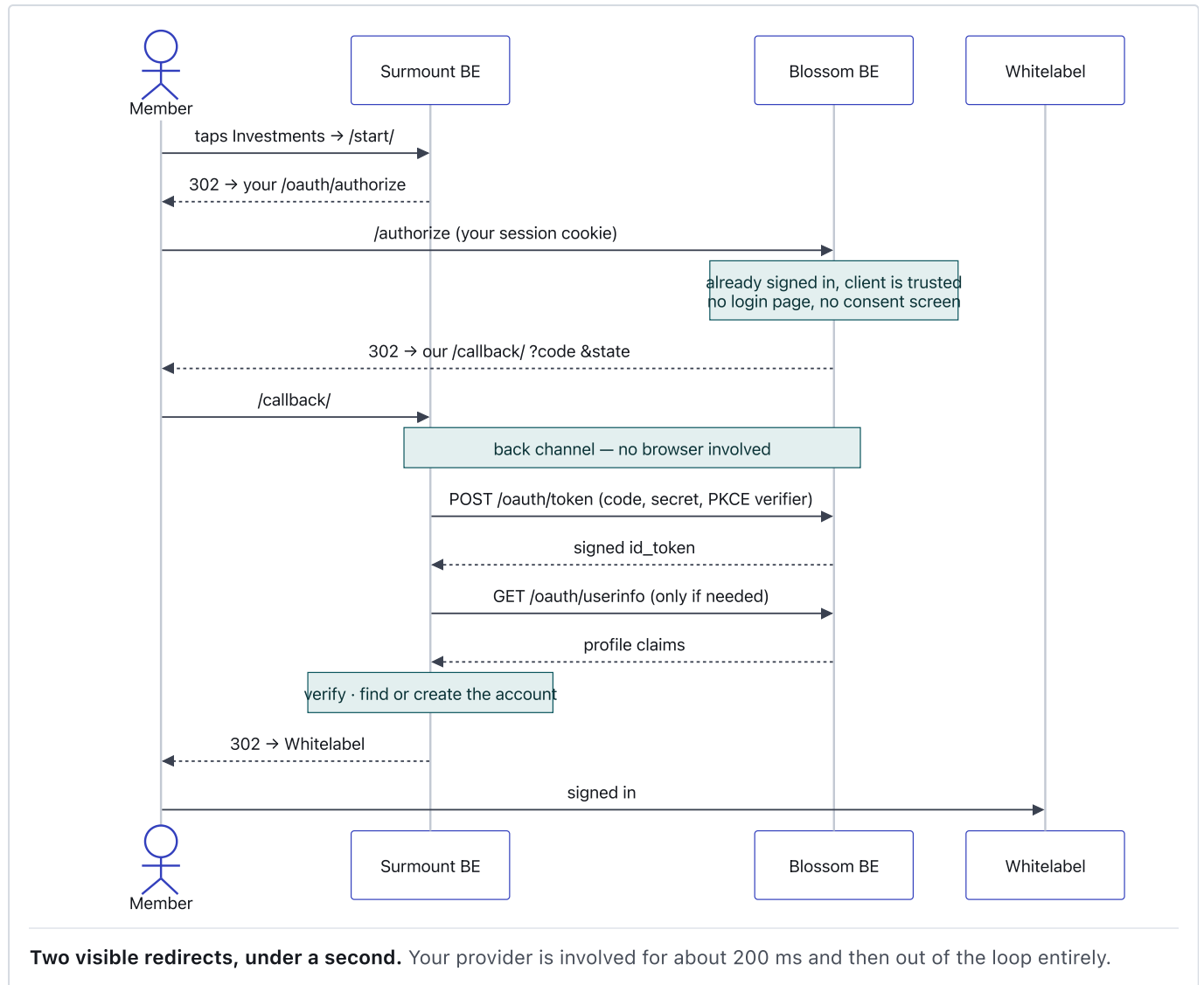
- Your frontend makes no API call.
- Your backend mints nothing and holds nothing.
- If a member never taps Investments, no request is ever made.

OPTIONAL: LAND THEM ON A SPECIFIC PAGE

Append `?next=/portfolio`. We accept it only as a path on our own app — an absolute URL is dropped.

07 How the flow works

You will not implement any of this — your library does. It is here so you can debug it.



WHAT EACH OF YOUR ENDPOINTS DOES

YOUR ENDPOINT	CALLED BY	DOES
<code>/.well-known/openid-configuration</code>	Our server	Tells us every other endpoint. Read once, cached an hour
<code>/.well-known/jwks.json</code>	Our server	Your public keys, for verifying signatures
<code>/oauth/authorize</code>	The member's browser	Reads your session cookie, issues an authorization code
<code>/oauth/token</code>	Our server	Trades the code for tokens. Client secret + PKCE verifier required
<code>/oauth/userinfo</code>	Our server	Profile claims, only if your ID token is minimal

IF THE MEMBER IS NOT SIGNED IN TO BLOSSOM

- Your `/authorize` endpoint has no session cookie to work with.
- It should show **Blossom's own login page**, preserving the original request (typically `?next=`).
- After signing in, the browser continues to `/authorize` and the flow completes.
- **This is what makes it single sign-on.** The member's password is only ever typed on Blossom's domain — never on ours.

OUR FLOW WINDOW IS ONE HOUR

Plenty of time for a password and an MFA prompt. You do not need to rush the member through your login page.

08 Test it before we meet

Run through this against your own provider. If every box ticks, the integration will work first time and we can skip a debugging call.

- ☐ **Discovery responds.** `curl {issuer}/.well-known/openid-configuration` returns 200 and valid JSON.
- ☐ **The document advertises your own issuer.** The `issuer` field matches the URL you gave us, exactly — including scheme and any trailing slash.
- ☐ **It advertises S256** in `code_challenge_methods_supported`.
- ☐ **It advertises RS256** in `id_token_signing_alg_values_supported`, and does **not** advertise `none`.
- ☐ **JWKS responds** at the advertised `jwks_uri`, contains at least one key with a `kid`, and contains **no private material** (no `d`, `p`, `q`, `dp`, `dq`, `qi` fields).
- ☐ **An unregistered redirect URI is refused** — and the error is **not** redirected to it.
- ☐ **A request without `code_challenge` is refused.**
- ☐ **An authorization code cannot be redeemed twice.**
- ☐ **A wrong PKCE verifier is refused** with `invalid_grant`.
- ☐ **A wrong client secret is refused** with `401`.
- ☐ **The ID token contains** `sub`, `email`, `email_verified`, `nonce`, and a `kid` in its header.
- ☐ **The `nonce` we sent is echoed back** inside the signed ID token.
- ☐ **A signed-in member is redirected straight through** — no consent screen for the trusted client.
- ☐ **A signed-out member sees Blossom's login page**, and completes the flow after signing in.

DEBUGGING TIP

Almost every first-run failure is one of three things: a redirect URI that differs by a trailing slash, an `issuer` in the discovery document that does not match the URL it was fetched from, or a client that is not marked trusted so it stalls on a consent screen the member never sees.

09 What you do not need to build

These are all handled on our side. Please do not scope them:

- × **Account creation on our side.** A member's first arrival creates their investing account silently.
- × **Linking a member who already has a Surmount account.** We detect and adopt it.
- × **A signup form, a password, or an email confirmation** in the investing app.
- × **Phone verification.** Our KYC flow collects and OTP-verifies it later, at the right moment.
- × **Retries, expiry handling, or error screens** for the hand-off.
- × **Any webhook or callback** back into Blossom.
- × **Passing us account balances, transactions or any banking data.** We only need identity.

10 What we do on our side

For your security review. Every step below is on the Surmount side.

1. **Starting the flow.** We generate `state`, `nonce` and a PKCE `code_verifier`, store all three server-side, and put only a random handle in an `HttpOnly`, `SameSite=Lax` cookie. Only the **SHA-256 hash** of the verifier goes in the URL.
2. **On the callback.** We compare `state` in constant time against the stored value **before any network call**. A mismatch stops everything — that is the login-CSRF defence.
3. **Exchanging the code.** Back-channel POST to your token endpoint with the code, our client secret in an `Authorization: Basic` header, and the PKCE verifier. Three independent proofs; stealing any one is not enough.
4. **Verifying the ID token.** Six checks against your JWKS: signature, `iss` exact match, `aud` equals our client ID, `exp` and `iat` with 60 s of clock skew, `nonce` match, and an algorithm allow-list that refuses `none` and `HS256`.
5. **Cross-checking `/userinfo`**. If we call it, its `sub` must match the ID token's. If they disagree, we accept neither.
6. **Single use.** The flow record is deleted on use, so a callback URL sitting in browser history is worth nothing.
7. **Our own session.** Never placed in a redirect URL. The final redirect carries a single-use handle with a 60-second life, traded for tokens over a POST.
8. **Audit logging.** Every hand-off decision — minted, verified, adopted, created, refused and why — is written to an audit log.

WHAT WE ASK YOU TO CONFIRM IN WRITING

- `sub` is immutable and never reused, including after user deletion.
- `email_verified: true` means the address was actually verified.
- A contact and process for **revoking a member's access** — see §12.

11 Frequently asked questions

Does Surmount ever see a member's Blossom password?

No. It is only ever typed on Blossom's own login page, on Blossom's domain. We never receive it, and there is no field in the protocol that could carry it.

What data crosses the boundary?

Only what is listed in §4: a subject identifier, an email address, a name and optionally an avatar URL. No balances, no transactions, no account numbers.

How long does our access token give Surmount access?

Five minutes, and we use it at most once, to read `/userinfo`. Your authorization code lives 60 seconds. Everything you issue is spent within a second of the hand-off.

Can we revoke a member's access?

Not today, and this is the one gap worth planning for. After the hand-off, Surmount runs its own session. We are implementing **OIDC Back-Channel Logout** so that ending a session at Blossom ends it at Surmount — if your library supports it, please enable it. In the meantime we can expose a simple client-authenticated revocation endpoint for your fraud team.

What happens if a member changes their email at Blossom?

Nothing breaks. We match on `sub`, which did not change. The new address is picked up on their next hand-off.

What if we rotate our signing key?

Publish the new key in your JWKS alongside the old one and start signing with it. We follow the `kid` header and refetch automatically. No downtime and no coordination needed — this is exactly why the key is asymmetric.

What if we rotate the client secret?

That one is coordinated — both sides at once. Give us a few days' notice. This is why the secret should be the less-frequently-rotated credential.

What if our provider is down?

Members see "try again" rather than an error page. We cache your discovery document and JWKS for an hour, so a brief outage does not stop members who are already mid-flow.

Does this work in the Blossom mobile app?

It needs a decision together. Embedded webviews (WKWebView, Android WebView) often do not share cookies with the system browser, which breaks the redirect chain. The correct approach is `ASWebAuthenticationSession` on iOS and Chrome Custom Tabs on Android, per RFC 8252 — and we would register a second redirect URI for it. Please raise it early if mobile is in scope.

Can a member start from the investing app instead?

Yes. A "Continue with Blossom" button on our side uses the same start URL. One endpoint serves both directions.

Do we need to tell members that Surmount exists?

Yes, and it is a regulatory requirement in most brokerage jurisdictions. "Investing is provided by Surmount" on the Investments surface, plus a mention in Blossom's privacy notice. See §12.

12 Before go-live

The technical integration is small. These are the non-technical items that take longer, so start them in parallel.

ITEM	OWNER	WHY
Data-sharing agreement	Legal, both sides	Blossom and Surmount are separate legal entities. Personal data moving between them needs a lawful basis and a documented controller relationship. A "trusted client" flag is not a substitute
Privacy notice update	Blossom	Members should be able to learn that their name and email go to Surmount, and why
"Provided by Surmount" disclosure	Blossom	On the Investments surface. Regulatory requirement in most brokerage jurisdictions
sub immutability, in writing	Blossom engineering	We build a permanent link on it. A reused sub is an unrecoverable failure
email_verified guarantee, in writing	Blossom engineering	We skip our own verification when we see true
Revocation process	Both	What happens when Blossom offboards or fraud-flags a member. See §11
Support runbook	Both	Blossom's support team will field "Investments says that took too long". They need to know what to tell members
Mobile scope decision	Both	Whether the link ships in the Blossom app, and how. See §11

Summary of what we need from you

- ☐ Your **issuer URL**
- ☐ A **client ID**
- ☐ A **client secret**, sent via a secrets manager
- ☐ Our redirect URI **allow-listed exactly**: `https://api.surmount.ai/api/sso/blossom/callback/`
- ☐ The client marked **trusted** (consent skipped)
- ☐ `sub` and `email` in the ID token, ideally with `email_verified`, `given_name`, `family_name`
- ☐ The **link** in your sidebar
- ☐ The §08 checklist ticked

Investments SSO · Blossom Developer Guide · OpenID Connect, Authorization Code with PKCE. Questions on any of the above are welcome before you start — most of them are faster to answer than to discover.